Echo Knight

1. Project Overview

Echo Knight is a 2D side-scrolling roguelike platformer built in Pygame, inspired by Hollow Knight. Players control a procedurally varied knight navigating through map levels, battling consistent enemies per stage. Core mechanics include precise platforming, combat, permadeath runs with character upgrades, and stat tracking for analysis. The game emphasizes tight controls, replayability through randomization, and data-driven insights into player performance.

2. Project Review

I got both inspiration from VictorHachard/pygame-roguelike on GitHub , a top-down turn-based roguelike using Pygame with tile-based maps and basic combat. Improvements in Echo Knight include shifting to side-scrolling platforming for fluid movement (real-time physics over turns), adding animated sprites like Hollow Knight-style (I make my own sprite with Piskel), so for this project I want to combine a rougelike playstyle into a side scrolling gameplay, the rougelike will focus on player character who will meet and get different randomly things around the map, some might be powerup, and some are skill.

VictorHachard/pygame: https://github.com/VictorHachard/pygame-roguelike

3. Programming Development

3.1 Game Concept

Echo Knight challenges players to traverse dangerous caves as a knight, reaching the stage exit while defeating enemies and collecting echoes (upgrades). Key mechanics: side-scrolling movement (run, jump, dash), melee combat (Punch), permadeath resetting the run but unlocking persistent meta-progression. Selling points: Roguelike replayability with procedural elements.

3.2 Object-Oriented Programming Implementation

- Player: Manages hero state/animations. Attributes: position, health (int), spritesheet (Surface). Methods: update() (physics/input), animate(state) (idle/run/jump), attack().

- Enemy: Handles AI foes (patrol/chase). Attributes: position, health, ai_state (str), patrol_points (list). Methods: update(player) (pathfinding), take_damage(), drop_echo().

- Map: Renders levels/collisions. Attributes: tiles (2D list), tile_size (int), tileset (dict). Methods: generate_procedural(width, height), get_collisions(rect).

- Camera: Smooth follows player. Attributes: offset (Vector2), bounds (Rect). Methods: update(player_rect), world_to_screen(pos).

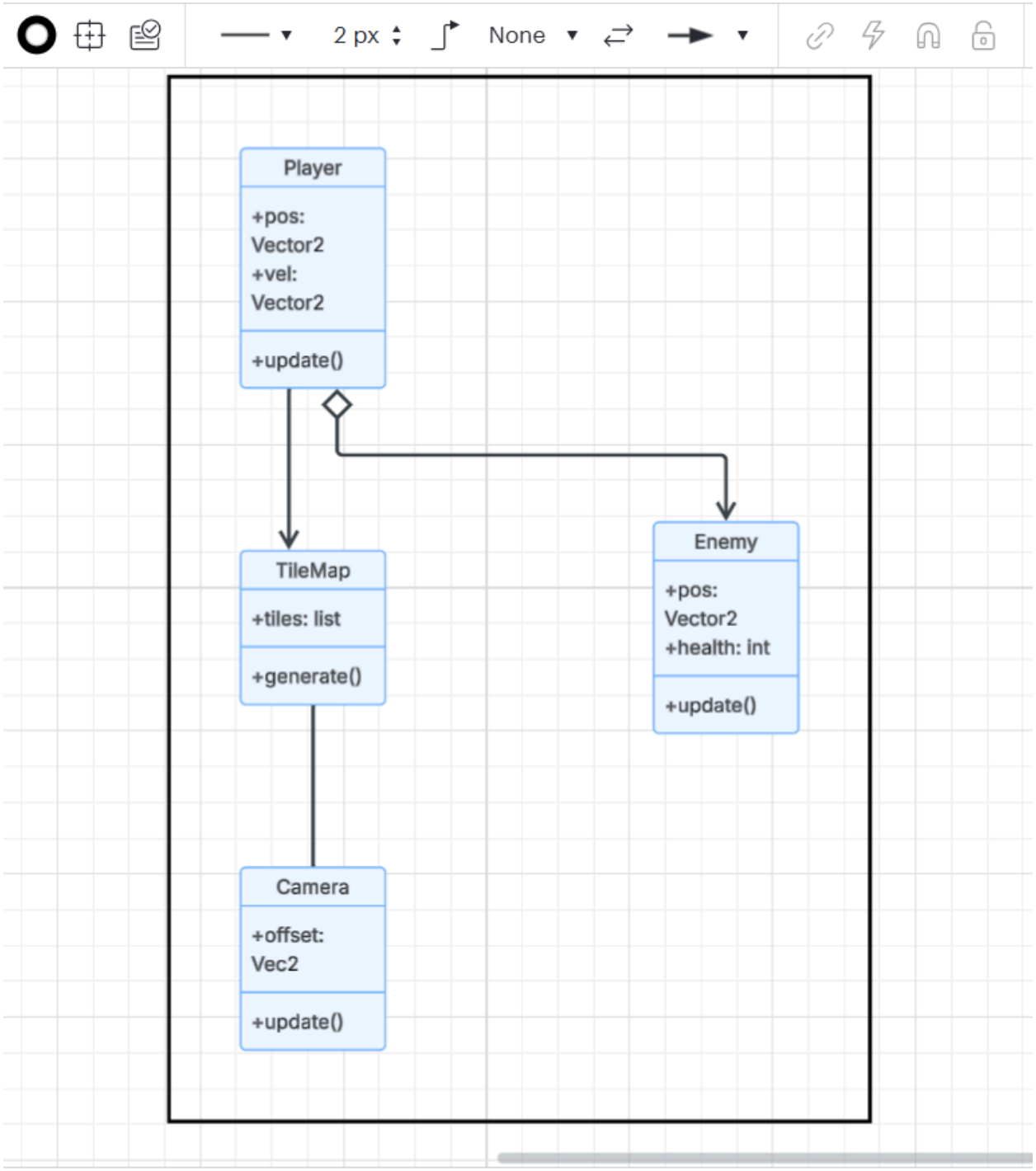
- GameStats: Tracks/logs metrics. Attributes: data (dict), csv_file (str). Methods: log_event(type, value), save_csv(), generate_report().

- GamePlay: This class will handle the game run and quit as well as handle the all entities for the game handle map making in the game.

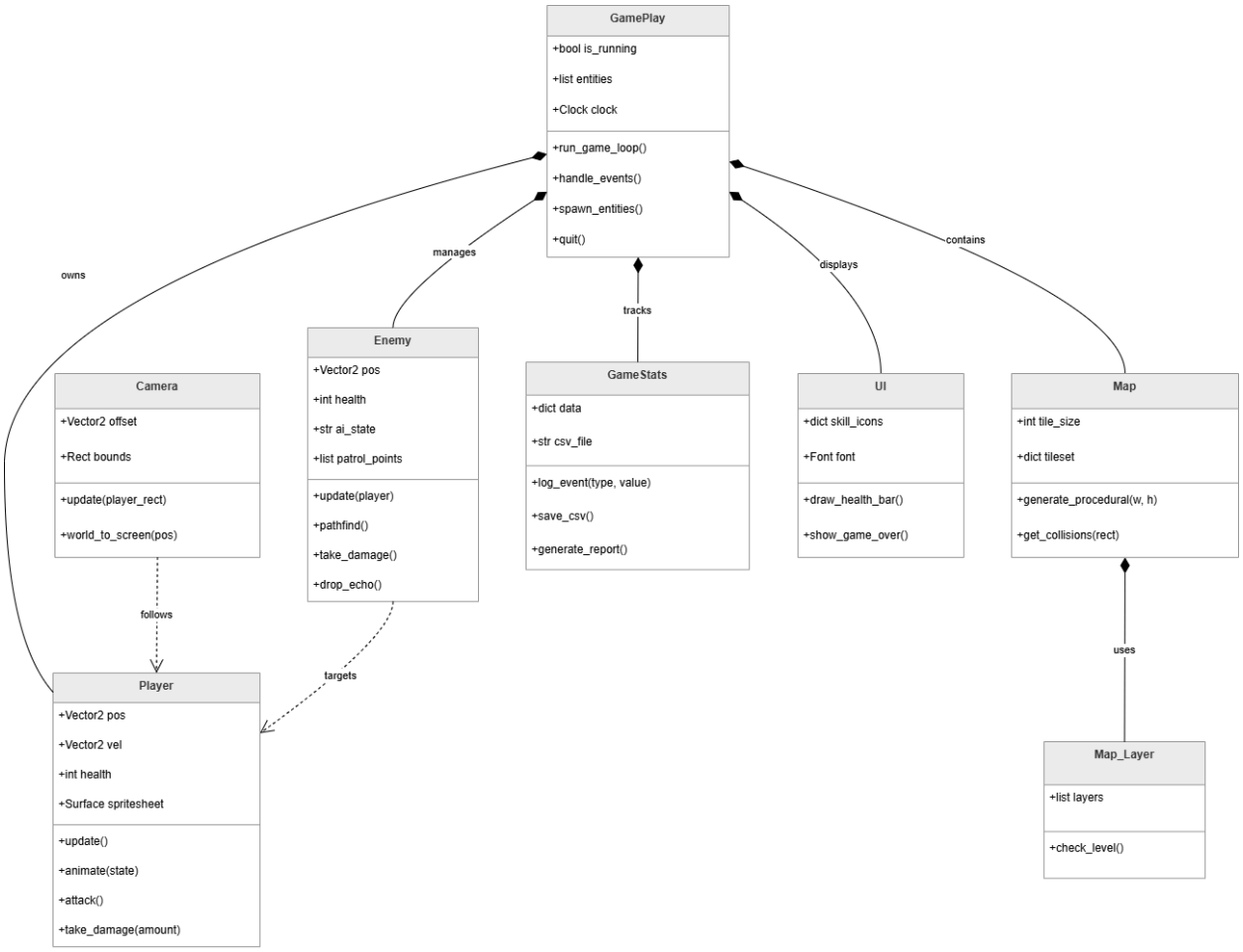
- UI: This class will handle the UI part for the health, Some skill Icon and the UI of game start and game end.

- Map_Layer: This class will handle layers of the map and also handle level of map level for checking and making the correct obstacle for the map.

First UML Design:



Second UML Design(26/3/2569):



3.3 Algorithms Involved

- Pathfinding: A* for enemy chase (grid-based, low-res for performance).
- Collision Detection: Axis-Aligned Bounding Box (AABB) for player/enemy/tiles.
- Animation Blending: State machine with frame timers for smooth transitions.
- Event-Driven Stats: Rule-based logging on ticks (e.g., every 60 FPS frames).

4. Statistical Data (Prop Stats)

4.1 Data Features

- Distance Traveled: Pixels moved horizontally (tracks exploration efficiency).
- Enemies Defeated: Count per run (with timestamps).
- Damage Taken: Total + per-hit sources (enemy/spike/fall).
- Run Time & Score: Seconds survived, echoes (Items) collected * multiplier.
- Keystrokes: Inputs per second (WASD/space/attack).
- Death Cause: String log (e.g., "enemy_hit", "fall").

4.2 Data Recording Method

Save to CSV file (game_stats.csv) using Python's csv module—append rows on events (e.g., writer.writerow([timestamp, 'distance', pixels])). Columns: timestamp, run_id, metric_type, value. Persistent across sessions for aggregation.

4.3 Data Analysis Report

Use Pandas (via code_execution if needed) for means, medians, correlations (e.g., jumps vs. survival time). Measures: averages, std dev, histograms. Present as tables/charts (Matplotlib PNGs): e.g., bar chart of death causes, line plot of distance over time. Export report to PDF.

Feature Name	Why is it good to have?	How to obtain 100 values?	Which variable/class?
Distance Traveled	Tracks player exploration and movement patterns.	Log pixels moved every 1 second of gameplay.	Player.position

Enemies Defeated	Measures combat difficulty and player pattern.	Record as "1" for every time an enemy health reaches 0.	Enemy.health (For checking health of enemies) GameStats (This for containing stat of enemies defeated)
Damage Sources	Identifies if spikes, falls, or enemies are the biggest threat.	Log the damage_type string every time take_damage() is called.	Player.take_damage()
Run Survival Time	Core metric for roguelike "skill" progression.	Log total seconds at the end of every run.	GamePlay.clock

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Survival TIme	Run Survival time	To look at how difficult of the game along with run time pattern behavior of player	Histogram Bar graph	Time Intervals	Number of Run
Distance Vs Enemies Defeated	Enemies defeated Distance traveled	Compare the run against the enemies that get defeated in that run	Scatterplot	Total Pixel moved	Total Kills

Death Cause	Damage sources	This is for checking level design and enemies with obstacle difficult	Pie Graph	-	-

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation, Base game
2	9	Combat basic, Enemies
3	10	Collision, Map variant, More Animation
4	11	Rougelike loop
5	12	Adding more Mechanic / Update progress through level
6	13	Improving UI / Bug Fix
7	14	Submission week (Draft)

6. Document version

Version: 1.5
Date: 27 March 2025
Editor: Picha Wiwattanawongsa

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	Be more specific with data presentation The class look pretty good but i dont think the UML is correct